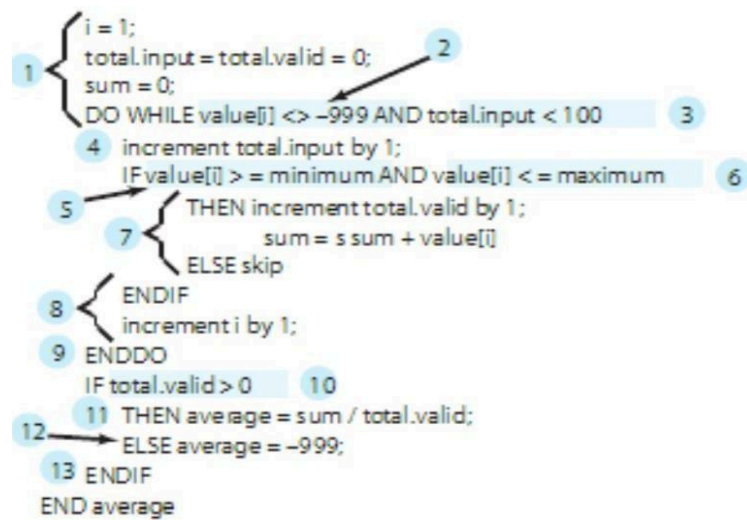


White Box Testing



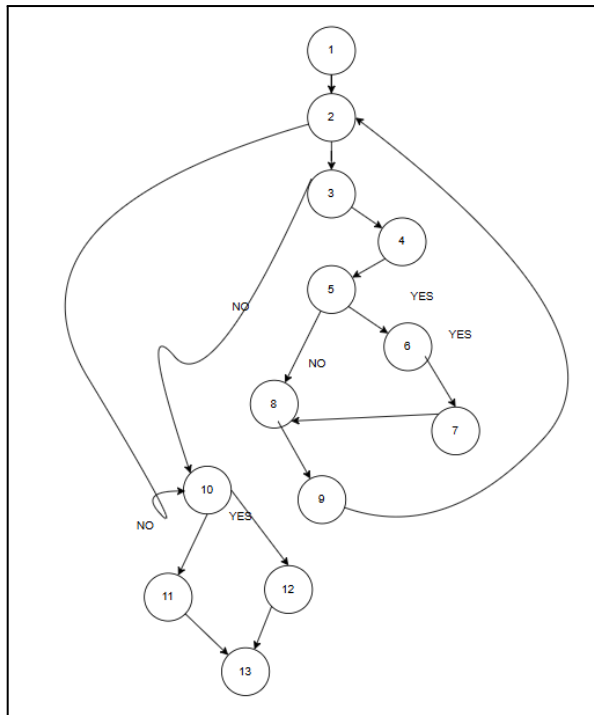
Source Code:

```

1  #include <iostream>
2  using namespace std;
3
4  const int MINIMUM = 0;
5  const int MAXIMUM = 100;
6
7  int main() {
8      int value[101];
9      int i = 1, total_input = 0, total_valid = 0;
10     double sum = 0;
11
12     // Input values
13     cout << "Enter values (enter -999 to stop input):\n";
14     for (i = 1; i <= 100; i++) {
15         cin >> value[i];
16         if (value[i] == -999) {
17             break;
18         }
19         total_input++;
20
21         if (value[i] >= MINIMUM && value[i] <= MAXIMUM) {
22             total_valid++;
23             sum += value[i];
24         }
25     }
26
27     double average;
28     if (total_valid > 0) {
29         average = sum / total_valid;
30     } else {
31         average = -999;
32     }
33
34     cout << "Total Input: " << total_input << endl;
35     cout << "Total Valid: " << total_valid << endl;
36     cout << "Average: " << average << endl;
37
38     return 0;
39 }

```

Control Flow Diagram:



Cyclomatic Complexity:

$$V(G) = E - N + 2$$

$$V(G) = 17 - 13 + 2$$

$$V(G) = 6$$

Linearly Independent Path :

Path 1: 1-2-10-11-13

path 2: 1-2-10-12-13

path 3: 1-2-3-10-11-13

path 4: 1-2-3-4-5-8-9-2-.....

path 5: 1-2-3-4-5-6-8-9-2-.....

path 6: 1-2-3-4-5-6-7-8-9-2-.....

Test Case Summary:

• Description:

Project Name:	Rod Cutting
Test Suite ID:	Calculate_TESE061
Test Title:	Test the Program Flow
Test Priority:	Medium
Module Name:	
Designed By:	Urwa Qadir
Designed Date:	1/2/2025
Executed By:	Urwa Qadir
Executed Date:	1/2/2025
Description of Test:	To Check The Rod Cutting Behaviour On different output

Test Cases:

SNO	Test Case	Test Steps	Test Data	Expected output	Actual Output	Pass/ Fail
TC_01	No values processed, directly checking total.valid	Enter The Value and For End enter -999	[-999]	Average :-999	Average :-999	Pass
TC_02	No valid values, only invalid ones	Enter The Value and For End enter -999	[105, 200, -999]	Average : -999	Average :-999	pass
TC_03	Loop starts but exits immediately	Enter The Value and For End enter -999	[50, -999]	Average : 50	Average :50	pass
TC_04	Loop runs, values are invalid, and then exits	Enter The Value and For End enter -999	[150, 200, -999]	Average :-999	Average :-999	pass
TC_05	Loop runs, mix of valid and invalid values	Enter The Value and For End enter -999	[20, 150, 50, -999]	Average : 35	Average :35	pass
TC_06	Loop runs, all values valid	Enter The Value and For End enter -999	[10, 20, 30, -999]	Average : 20	Average : 20	pass

Output:

path1:

```

Enter values (terminate with -999):
-999
Average: -999

-----
Process exited after 2.795 seconds with return value 0
Press any key to continue . . .

```

path 2:

```

Enter values (terminate with -999):
105 200 -999
Average: -999

-----
Process exited after 20.34 seconds with return value 0
Press any key to continue . . .

```

path 3:

```

Enter values (enter -999 to stop input):
50 -999
Total Input: 1
Total Valid: 1
Average: 50

-----
Process exited after 8.494 seconds with return value 0
Press any key to continue . . .

```

Path 4:

```

Enter values (terminate with -999):
150 200 -999
Average: -999

-----
Process exited after 14.47 seconds with return value 0
Press any key to continue . . .

```

Path 5:

```

Enter values (enter -999 to stop input):
20 150 50 -999
Total Input: 3
Total Valid: 2
Average: 35

```

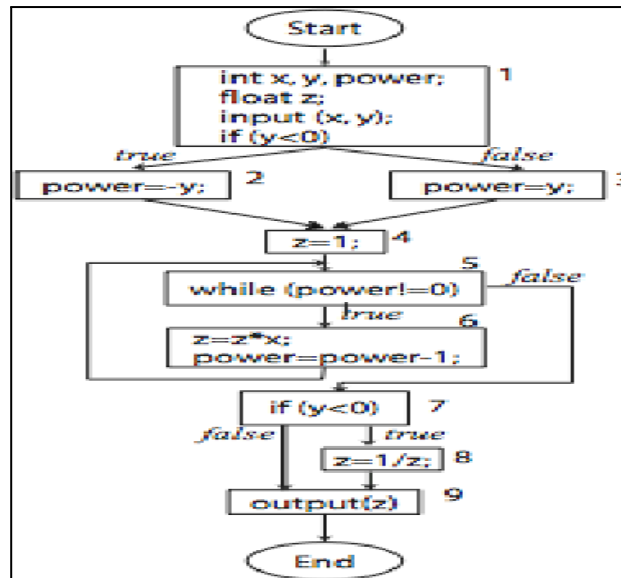
path 6:

```

Enter values (enter -999 to stop input):
10 20 30 -999
Total Input: 3
Total Valid: 3
Average: 20

```

1. Consider the flow chart given below and design test cases by using statement coverage method, AND Independent path method also execute test cases using template (Attach All printout here).



Code:

```

def main():
    x = int(input('enter value of x : '))
    y = int(input('enter value of y : '))
    if y < 0:
        power = -y
    else:
        power = y
    z = 1
    while (power!=0):
        z = z*x
        power = power -1
    if y<0:
        z = 1/z
    print(f"result: {z}")

if __name__ == "__main__":
    main()

```

TEST SUMMARY REPORT

Project Name	Power_Calculator
--------------	------------------

Test suite ID	Calculate_TESE22099
Test Title	Test the power operation
Test priority	Medium
Module Name	Power_Calculator
Designed by	Jassica
Designed Date	27-Jan-25
Executed by	Jassica
Executed Date	27-Jan-25
Description of Test	Verify the power calculation based on user input (y < 0, y=0, y>0)
PREREQUISITES	
Pre-Condition	Not required
Dependencies	No dependencies

Statement Coverage

Test Cases:

Sno	test case	test steps	test data	expected output	actual output	pass/fail	comments
TC_01	to validate the statements executed in y < 0	input x and y	x = 12 y = -4	0.0000482	0.0000482	pass	
TC_02	to validate the statements executed in y > 0	input x and y	x = 12 y = 4	20736	20736	pass	
TC_03	to validate the statements executed in y = 0	input x and y	x = 0 y = 0	1	1	pass	as initially z =1 so output will be 1

Test Case Execution

TC_01 : executing all if statements (true) 1, 2, 3, 5, 6, 7, 8, 9, 10

Coverage = (9/10) *100 = 90%

```
PS C:\Users\ALLEN\OneDrive\Desktop\exercise\Microsoft VS Code
enter value of x : 12
enter value of y : -4
result: 4.8225308641975306e-05
```

TC_02 : executing all else statements (false) 1, 2, 4, 5, 6, 7, 10

Coverage = $(7/10) \times 100 = 70\%$

```
PS C:\Users\ALLEN\OneDrive\Desktop\exercise\Microsoft VS Code
enter value of x : 12
enter value of y : 4
result: 20736
```

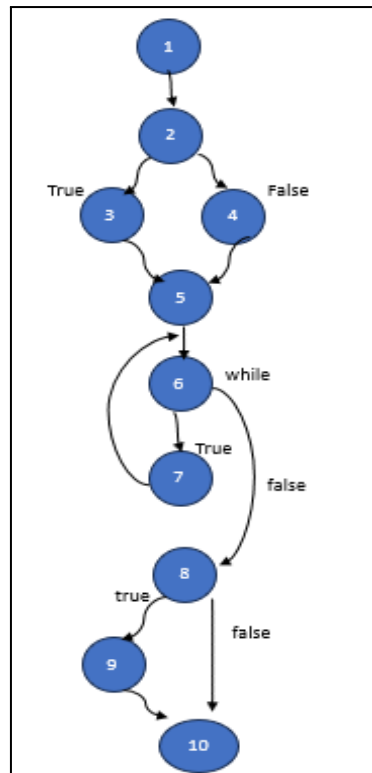
TC_03 : validating while loop with y = 0 1, 2, 4, 5, 6, 10

Coverage = $(6/10) \times 100 = 60\%$

```
PS C:\Users\ALLEN\OneDrive\Desktop\exercise\Microsoft VS Code
enter value of x : 0
enter value of y : 0
result: 1
```

Linearly Independent Path

CFG



Cyclomatic Complexity:

$$V(G) = E - N + 2$$

$$V(G) = 12 - 10 + 2$$

$$V(G) = 4$$

Independent Path :

Path 1: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 6 \dots \rightarrow 10$

path 2: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \dots \rightarrow 6 \rightarrow 8 \rightarrow 9 \rightarrow 10$

path 3: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 10$

path 4: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 6 \rightarrow 8 \rightarrow 9 \rightarrow 10$

TEST CASES:

Sno	test case	test steps	test data	expected output	actual output	pass/fail
TC_01	to validate the path executed in $y < 0$	input x and y	$x = 12 \quad y = -4$	0.0000482	0.0000482	pass
TC_02	to validate the path executed in $y > 0$	input x and y	$x = 12 \quad y = 4$	20736	20736	pass
TC_03	to validate paths with $y = 0$	input x and y	$x = 3 \quad y = 0$	1	1	pass
TC_04	validte with $x = 1$	input x and y	$x = 1 \quad y = -3$	1	1	pass

Path 1:

```
PS C:\Users\ALLEN\OneDrive\Desktop\exerc
enter value of x : 12
enter value of y : 4
result: 20736
PS C:\Users\ALLEN\OneDrive\Desktop\exerc
```

Path 2:

```
PS C:\Users\ALLEN\OneDrive\Desktop\exercise\Microsoft VS Code
enter value of x : 12
enter value of y : -4
result: 4.8225308641975306e-05
PS C:\Users\ALLEN\OneDrive\Desktop\exercise\Microsoft VS Code
```

Path 3 :

```
PS C:\Users\ALLEN\OneDrive\Desktop\exercise\Microsoft VS Code
enter value of x : 3
enter value of y : 0
result: 1
PS C:\Users\ALLEN\OneDrive\Desktop\exercise\Microsoft VS Code
```

Path 4 :

```
PS C:\Users\ALLEN\OneDrive\Desktop\exercise\Microsoft VS Code
enter value of x : 1
enter value of y : -3
result: 1.0
PS C:\Users\ALLEN\OneDrive\Desktop\exercise\Microsoft VS Code
```

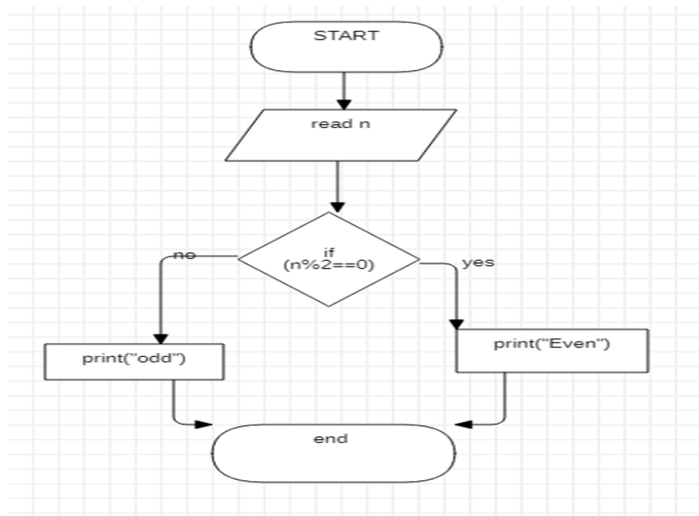
2. Write a program (Any Language) to find odd or even number. Design and execute test cases using branch coverage method using template /*attach printout here*/

Source Code:

```
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout<<"Enter a number";
    cin>> n;

    if(n%2==0){
        cout<<"Even";
    }
    else{
        cout<<"Odd";
    }
    return 0;
}
```

Flow Chart:



Test Cases:

Statement Coverage

tc_ID	test case	test step	test data	expected	actual	pass/fail
tc_001	check if number is even	1. enter number	n = 8	even	even	Pass
tc_002	check if number is odd	1. enter number	n = 7	odd	odd	Pass

Output:

```
Enter a number: 8
Even
-----
Process exited after 2.769 seconds with return value
Press any key to continue . . .
```

```
Enter a number: 7
Odd
-----
Process exited after 2.596 seconds with return value
Press any key to continue . . .
```

Branch Coverage:

tc_ID	test case	test step	test data	expected	actual	pass/fail
tc_001	check if number is even	1. enter number	n = 8	even	even	pass
tc_002	check if number is odd	1. enter number	n = 7	odd	odd	pass

Output:

Test Case 1:

```
Enter a number: 8
Even
-----
Process exited after 2.769 seconds with return value
Press any key to continue . . .
```

Test Case 2:

```
Enter a number: 7
Odd
-----
Process exited after 2.596 seconds with return value
Press any key to continue . . .
```

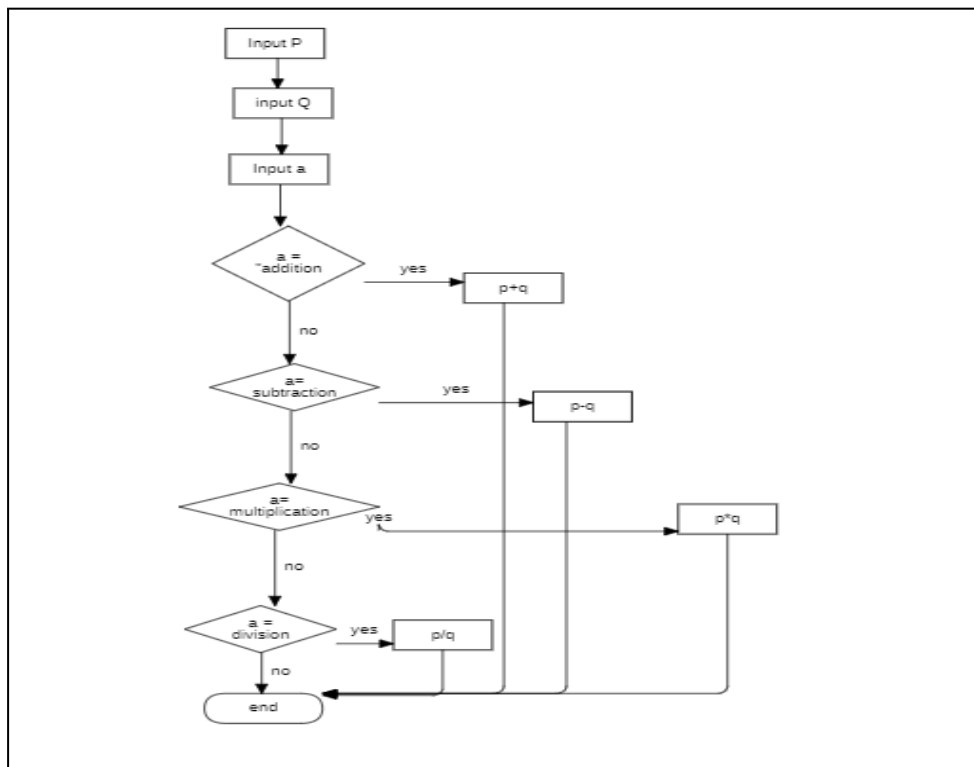
3. Write a program (Any Language) to perform addition, subtraction,

multiplication and division. Observe the output and Test it using any one of white box technique .Attached all printout here.

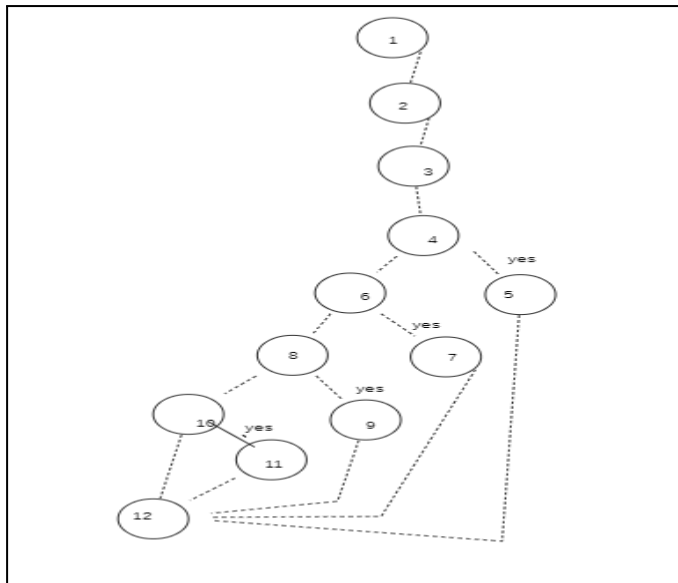
Source Code:

```
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      int p;
6      int q;
7      cout<<"Enter first number";
8      cin>>p;
9      cout<<"Enter second number";
10     cin>>q;
11
12     cout<<"Select operation 1-Addition, 2 subtraction, 3-multiplication, 4- division \n";
13     int a;
14     cin>>a;
15     if (a==1){
16         cout<< p+q;
17     }
18     if (a==2){
19         cout<< p-q;
20     }
21     if (a==3){
22         cout<< p*q;
23     }
24     if (a==4){
25         cout<< p/q;
26     }
27
28     return 0;
29 }
```

Flow chart:



CFG:



Possible paths:

path 1: 1-2-3-4-5-12

path 2: 1-2-3-4-6-7-12

path 3: 1-2-3-4-6-8-12

path 4: 1-2-3-4-6-8-10-11-12

path 5: 1-2-3-4-6-8-10-12

Test Cases:

s.no	Test case	test steps	test data	expected output	actual output	pass/fail
1	Verify addition	1. enter p, 2. enter q, 3. choose addition	p=4, q=2, operator = add	6	6	pass
2	Verify subtraction	1. enter p, 2. enter q, 3. choose subtraction	p=2, q=4, operator = subtract	-2	-2	pass
3	Verify multiplication	1. enter p, 2. enter q, 3. choose multiplication	p=9, q=4, operator = multiplication	36	36	pass
4	Verify division	1. enter p, 2. enter q, 3. choose division	p= 20, q=5, operator=divide(4)	4	4	pass

5	Verify number chosen for operator	1. enter p, 2. enter q, 3. choose other number	p= 20, q=5, operator=7	no result	no result	pass
---	-----------------------------------	--	------------------------	-----------	-----------	------

Output:

Test Case 1:

```

Enter first number4
Enter second number2
Select operation 1-Addition, 2 subtraction, 3-multiplication, 4- division
1
6
-----
Process exited after 13.18 seconds with return value 0
Press any key to continue . . .

```

Test Case 2:

```

Enter first number2
Enter second number4
Select operation 1-Addition, 2 subtraction, 3-multiplication, 4- division
2
-2
-----
Process exited after 4.455 seconds with return value 0
Press any key to continue . . .

```

Test Case 3:

```

Enter first number9
Enter second number4
Select operation 1-Addition, 2 subtraction, 3-multiplication, 4- division
3
36
-----
Process exited after 4.856 seconds with return value 0
Press any key to continue . . .

```

Test Case 4:

```

Enter first number20
Enter second number5
Select operation 1-Addition, 2 subtraction, 3-multiplication, 4- division
4
4
-----
Process exited after 5.581 seconds with return value 0
Press any key to continue . . .

```

Test Case 5:

```

Enter first number1
Enter second number2
Select operation 1-Addition, 2 subtraction, 3-multiplication, 4- division
9
-----
Process exited after 5.321 seconds with return value 0
Press any key to continue . . .

```